Chapter Two

Talking to Claude Well

Direct Claude clearly and you will pull dramatically better work out of the same model, no tricks required.

The gap between a mediocre answer and an excellent one is almost never the model. It is the instruction. Two people ask Claude the same kind of question on the same day, and one walks away with a polished, usable result while the other gets something vague and generic. The difference is that one of them told Claude exactly what they wanted and the other left it to guess.

Good news lives inside that fact. You do not need a secret phrase or a jailbreak or a thousand-word template to get expert output. You need to say what you want plainly, give the goal, and set the constraints. This chapter teaches that skill from the ground up, using Anthropic's own guidance, translated into moves you can make from any chat box.

By the end you will know how to state a task so it lands the first time, how to show Claude the shape of the answer you want, how to structure a longer prompt so the model reads it cleanly, when to let Claude think harder before it replies, and how to lock in a tone and format you reuse every day.

2.1 Clear direction beats clever tricks

Anthropic gives one mental model that does most of the work: treat Claude like a brilliant new employee on their first day. They are sharp and capable. They also have no idea what your norms are, what "done" looks like to you, or which details you care about. The more precisely you brief them, the better

the result. Leave things implied and you get a literal reading of a half-specified request.

Three habits carry most of the load. Be specific about the output you want, including its format and length. State the goal, so Claude understands what success means. Give the constraints up front, so it does not have to guess and guess wrong. When the order of steps matters, spell them out as a numbered list so Claude never has to reconstruct your sequence.

> **TIP**
>
> Anthropic's golden-rule test for any prompt: show it to a colleague who has minimal context on the task and ask them to follow it. If they would be confused about what to do, Claude will be too. This one check catches most weak prompts before you ever send them.

There is a second half to specificity that people miss. Tell Claude *why* a rule matters, and it will apply the rule intelligently instead of following it like a robot. A bare "never use ellipses" is a narrow order. Explain that the text will be read aloud by a text-to-speech engine that cannot pronounce them, and Claude generalizes the intent to every similar case you did not think to list. Motivation travels further than a command.

WORKED EXAMPLE · TURNING A WEAK PROMPT INTO A STRONG ONE

Say you want a dashboard mocked up as an artifact. The instinct is to type the shortest thing that names the task.

Weak prompt

```
Create an analytics dashboard
```

Claude will build something, and it will be thin, because you asked for the bare concept and nothing more. Now add the goal and the level of ambition you actually have in mind.

Strong prompt

```
Create an analytics dashboard. Include as many relevant features
and
interactions as possible. Go beyond the basics to create a fully-
featured
implementation.
```

Same model, same surface, one added sentence. The second version routinely produces a richer, more complete result, because Claude no longer has to guess how far you wanted it to go. You told it. That is the entire technique.

GOTCHA

Current Claude models follow you very literally, and that bites when you phrase an action as a question. "Can you suggest some changes?" gets you a list of suggestions, not the edits, even when you clearly meant for Claude to just do it. When you want action, use plain imperative phrasing: "Change this function to improve its performance" or "Rewrite this paragraph to be shorter." Ask for a suggestion and you get a suggestion.

One more phrasing habit pays off constantly. Tell Claude what to do instead of what to avoid. "Do not use markdown" is weaker than "Write your response as smoothly flowing prose paragraphs," because the positive version gives the model a target to hit instead of a fence to avoid. Whenever you catch yourself writing a "do not," try to flip it into the thing you do want.

QUICK REFERENCE · WEAK PHRASING AND ITS STRONGER FORM

Instead of	Write
Summarize this	Summarize this in three bullet points a busy executive could skim
Do not use markdown	Write your response as flowing prose paragraphs
Can you suggest edits?	Edit the draft directly for clarity and length
Make it better	Cut it to 200 words and make the opening line concrete

Never use jargon	Explain it so a smart 15-year-old would follow, since a beginner will read it

2.2 Show, do not only tell

Some things are hard to describe in words and easy to demonstrate. The exact tone of a reply email, the precise structure of a product blurb, the way you like edge cases handled: for all of these, an example is worth a paragraph of instruction. Anthropic calls giving worked examples one of the most reliable ways to steer Claude's output, and it costs you almost nothing.

KEY TERM

Few-shot prompting giving Claude a few worked examples of the task, each showing an input and the ideal output, so it learns the pattern by imitation instead of from description alone. "Few" is literal: three to five good examples is the sweet spot.

Good examples share three traits. They are relevant, meaning they mirror your real use case closely. They are diverse, meaning they vary enough to cover the edge cases and stop Claude from latching onto some accidental pattern the examples happen to share. And they are clearly marked, so Claude can tell an example apart from an instruction. In a chat, the simplest way to mark them is a plain label and a blank line, like "Example 1:" above each one.

TIP

You do not have to write the examples yourself. Paste one or two, then ask Claude to generate three more in the same style, or to check whether your examples are varied enough to be useful. It is happy to critique its own inputs, and this bootstrapping trick turns a single sample into a full set in one turn.

The other way to show instead of tell is to hand Claude the reference material directly. Rather than describing your brand voice or your data, paste the style guide, the source document, the spreadsheet, or the transcript, and ask Claude to work from it. When you do this with a long document, one habit matters more than any other: put the long material near the top of your

message, above the actual question. Claude answers noticeably better when the bulky reference comes first and your instruction comes last, especially when you have pasted several documents at once.

> **TIP**
>
> With a large pasted document, ask Claude to first pull out the quotes that are relevant to your question, then answer using only those quotes. Grounding the reply in specific passages keeps Claude focused on the material that matters and cuts down on it wandering into the noise of a long file. It is the single most effective move for long-document work.

2.3 Give a prompt a shape Claude can read

Once a prompt grows past a sentence or two, it starts mixing different kinds of content: your instruction, the background context, the document to work on, the examples to imitate. When all of that runs together as one blob, Claude has to guess where one part ends and the next begins, and it sometimes guesses wrong, treating your data as an instruction or your instruction as data. A little structure removes the ambiguity.

The plainest structure is headings. Break your message into labeled parts, like a short memo: a "Context" section, a "Task" section, an "Output format" section. Even that much helps Claude parse a busy request. For anything with a document or variable input embedded in it, Claude reads XML-style tags especially cleanly, because it has been tuned to recognize them.

A structured prompt

```
<context>
I run a small bakery. The tone should be warm and casual.
</context>

<task>
Write three Instagram captions for tomorrow's sourdough special.
</task>

<constraints>
Under 40 words each. Include one emoji. No hashtags.
</constraints>
```

Nothing about those tags is magic. They are just consistent, descriptive labels that fence each type of content off from the others. Use whatever names read clearly to you and keep them consistent: `<context>`, `<document>`, `<examples>`. When your input has natural layers, such as several documents, you can nest tags to match. The point is to make the boundaries obvious so Claude never has to infer them.

The same section that carries your context is also where you set a role. Telling Claude who it is at the top of the conversation measurably shapes everything that follows. "You are a careful copy editor who cuts every unnecessary word" produces different, better-aimed output than the same request with no framing at all. One sentence of role is one of the cheapest upgrades available to you, and it pairs naturally with the context block above.

2.4 When to let Claude think first

For a hard problem, an answer improves when Claude reasons through it before committing to a reply, the same way you would sketch on scratch paper before writing a final answer. Claude offers a mode for exactly this, usually called extended or adaptive thinking, where the model works through the problem step by step and then gives you its conclusion.

KEY TERM

Extended thinking a mode where Claude reasons through a problem internally before answering, which raises quality on tasks that need real multi-step reasoning. On current models Claude increasingly decides for itself when a question warrants it, scaling the effort to the difficulty of what you asked.

The useful judgement is knowing when thinking earns its keep. Reach for it when the task genuinely needs reasoning: a tricky logic or math problem, planning a multi-step piece of work, weighing several options against each other, debugging why something went wrong. On those, the extra deliberation shows up directly in the quality of the answer.

! GOTCHA

Thinking is overkill for simple work. A quick lookup, a short rewrite, a factual question with one clear answer: forcing deep reasoning on these buys you nothing and only makes the reply slower. The heuristic is plain. If the problem is one you could answer instantly yourself, Claude can too, and it does not need to think out loud to get there.

How you switch thinking on, and whether it is even a control you touch, depends on which app and which model you are using, and Anthropic keeps changing the specifics as models improve. The durable takeaway is the concept: harder questions benefit from more deliberation, easy ones do not, and newer models handle more of that decision on their own. For how to enable it on the model in front of you today, check the current thinking documentation instead of the setting you saw in an older guide.

I TIP

You can nudge thinking with plain words even without touching a setting. For a careful answer, add "work through this step by step before you answer." To catch mistakes on math or logic, end with "before you finish, double-check your answer against the requirements." A short verification instruction reliably surfaces errors the first pass missed.

2.5 Styles: lock in a tone once

Everything so far shapes one message. When you find yourself typing the same tone-and-format instructions at the start of every chat, promote them into a reusable style. A style is a saved set of instructions about how Claude should write for you: the voice, the formatting, the length, the things to always do and always avoid. You set it once and Claude applies it to every conversation you point at it, so you stop re-explaining your preferences each morning.

A style is the natural home for the format discipline from earlier in this chapter. If you always want flowing prose instead of bullet-point soup, or a warm first-person voice, or replies that skip the throat-clearing preamble and get straight to the point, those belong in a style. Write them the same way you would write any good instruction: say what to do, explain why when it helps, and be specific about the format you want.

QUICK REFERENCE · WHAT TO REACH FOR WHEN

If you want to	Reach for
Get a clearly-specified one-off task right	A specific prompt: goal, format, constraints
Nail an exact tone or structure that is hard to describe	A few examples (three to five)
Work against a long document or dataset	Paste it at the top, ask for quotes first
Keep instructions and content from blurring together	Headings or XML-style tags
Solve a hard, multi-step reasoning problem	Extended thinking or a "step by step" nudge
Apply the same voice and format across every chat	A saved style

Notice the through-line across the whole chapter. Every one of these moves is a way of removing a guess. A specific prompt removes the guess about what you want. Examples remove the guess about what good looks like. Structure

removes the guess about which words are the task. Thinking removes the guess that comes from answering too fast. A style removes the guess about your voice. Get in the habit of asking, before you hit enter, what is Claude still having to guess here, and your results climb without any cleverness at all.

CHAPTER 2 · CHECKLIST

You can now

- Brief Claude like a capable new hire: state the goal, the format, and the constraints up front.
- Run the colleague test on a prompt and fix it before sending if a person would be confused.
- Phrase actions as imperatives so Claude does the work instead of only suggesting it.
- Steer output with three to five relevant, varied examples when words alone fall short.
- Put long reference material at the top of a message and ask Claude to quote before it answers.
- Separate instructions from content with headings or XML-style tags, and set a role in one sentence.
- Judge when extended thinking helps a hard problem and when it is wasted on a simple one.
- Save a reusable style so your tone and format carry across every chat automatically.